

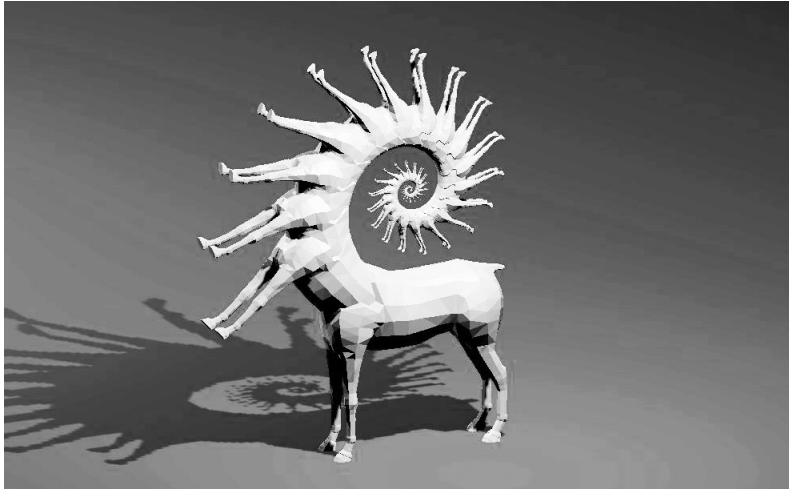


Figure 1-3: The recursive centaur. Image by Joseph Parker.

Based on these jokes, you might conclude that recursion is a sort of meta, self-referencing, dream-within-a-dream, infinite mirror-into-mirror sort of thing. Let's establish a concrete definition: a *recursive* thing is something whose definition includes itself. That is, it has a self-referential definition.

The Sierpiński triangle in Figure 1-4 is defined as an equilateral triangle with an upside-down triangle in the middle that forms three new equilateral triangles, each of which contains a Sierpiński triangle. The definition of Sierpiński triangles includes Sierpiński triangles.

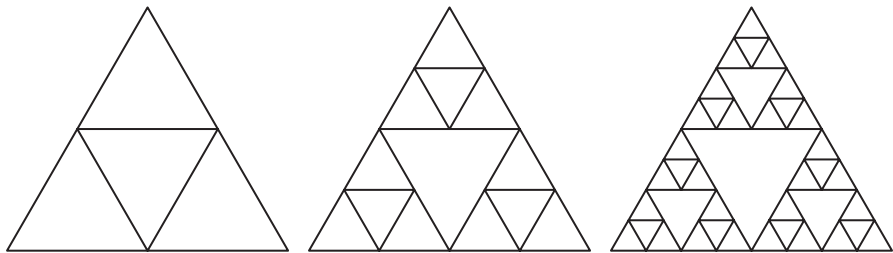


Figure 1-4: Sierpiński triangles are fractals (recursive shapes) that include Sierpiński triangles.

In a programming context, a *recursive function* is a function that calls itself. Before we explore recursive functions, let's take a step back and understand how regular functions work. Programmers tend to take function calls for granted, but even experienced programmers will find it worthwhile to review functions in the next section.

What Are Functions?

Functions can be described as mini-programs inside your program. They're a feature of nearly every programming language. If you need to run identical

```

function b() {
    document.write("b() was called.<br />");
    ❷ c();
    document.write("b() is returning.<br />");
}

function c() {
    document.write("c() was called.<br />");
    document.write("c() is returning.<br />");
}

a();
</script>

```

When you run this code, the output looks like this:

```

a() was called.
b() was called.
c() was called.
c() is returning.
b() is returning.
a() is returning.

```

The output shows the start of functions `a()`, `b()`, and `c()`. Then, when the functions return, the output appears in reverse order: `c()`, `b()`, and then `a()`. Notice the pattern to the text output: each time a function returns, it remembers which line of code originally called it. When the `c()` function call ends, the program returns to the `b()` function and displays `b() is returning..` Then the `b()` function call ends, and the program returns to the `a()` function and displays `a() is returning..` Finally, the program returns to the original `a()` function call at the end of the program. In other words, function calls don't send the execution of the program on a one-way trip.

But how does the program remember if it was `a()` ❶ or `b()` ❷ that called `c()`? This detail is handled by the program implicitly with a call stack. To understand how call stacks remember where the execution returns at the end of a function call, we need to first understand what a stack is.

What Are Stacks?

Earlier I mentioned the clichéd wisecrack, “To understand recursion, you must first understand recursion.” But this is actually wrong: to really understand recursion, you must first understand stacks.

A *stack* is one of the simplest data structures in computer science. It stores multiple values like a list does—but unlike lists, it limits you to adding to or removing values from the “top” of the stack only. For lists, the “top” is the last item, at the right end of the list. Adding values is called *pushing* values onto the stack, while removing values is called *popping* values off the stack.

Imagine that you're engaged in a meandering conversation with someone. You're talking about your friend Alice, which then reminds you of a